Pattern description and solution

This pattern has three sub-patterns, which work together. These are: circuit breaker, pool ejection, and request throttling.

Circuit breaker: Assuming the above happened in a distributed system, the solution is to implement service mesh circuit breaker (CB) and pool ejection patterns to limit the impact of such failures, latency spikes and other network irregularities. The pattern is to 'open' the circuit and/or eject unhealthy containers for a designated period. The CB can be at:

- Ingress point
- Between each service within the mesh
- At egress while calling an external service

Within the service mesh, *envoy proxy* can intercept calls and enforce a circuit breaking limit at the network level, as opposed to having it configured and coded for each application independently, thus supporting distributed circuit breaking.

Figure 1 depicts a logical view of setting up circuit breakers at the point of ingress. This is done by:

i. Configuring *ingress gateway* as a circuit breaker.
ii. Routing a request through ingress, detecting the behaviour of the traffic and protecting the service being called within the cluster.
iii. In case of failure, immediately returning the response to the consumer with a proper failure code.

Figure 2 depicts a logical view of setting up the circuit breaker between services inside the mesh. This is done by:

i. Configuring envoy proxy as a circuit breaker between services inside the mesh.
ii. Routing consumer requests by ingress gateway to envoy proxy in POD A.
iii. Service A calling to Service B in POD B via its envoy proxy.
iv. POD A's envoy proxy detecting the behaviour of the traffic that is protecting the service being called within the cluster as well as the consumer.

Figure 3 depicts a logical view of setting up the circuit breaker at the point of egress. This is done by:

i. Configuring envoy proxy as circuit breaker at the point of egress to an external host.
ii. Routing request from consumer request through egress gateway.
iii. Detecting behaviour of the traffic and protecting the external service or system of record.

Pool ejection: In pool ejection, outlier detection prevents calls to unhealthy services and ensures any call is routed to healthy service instances. It can be used in conjunction with CB to manage the behaviour of the response to an unhealthy service.

Figure 4 depicts a logical view of setting up outlier detection using pool ejection. This is done by:

i. Configuring envoy proxy as circuit breaker with outlier detection to eject unhealthy services until such point that a minimal health percentage is detected.
ii. At this point, as a second line of defence, the circuit breaker opens if the behaviour deteriorates further due to a breach of failure rules.

***Request throttling*:** Service mesh also provides the capability of request throttling (RT) by configuring RT at an individual client level so that no single client can consume all the available resources.

Figure 5 depicts a logical view of setting up request throttling. This is done by:

i. Setting up a global rate limiting filter for network level rate limit filter, with HTTP level rate limit filter set at the listener level.
ii. Setting up a local rate limiting filter at a more specific virtual host or route level of the service.

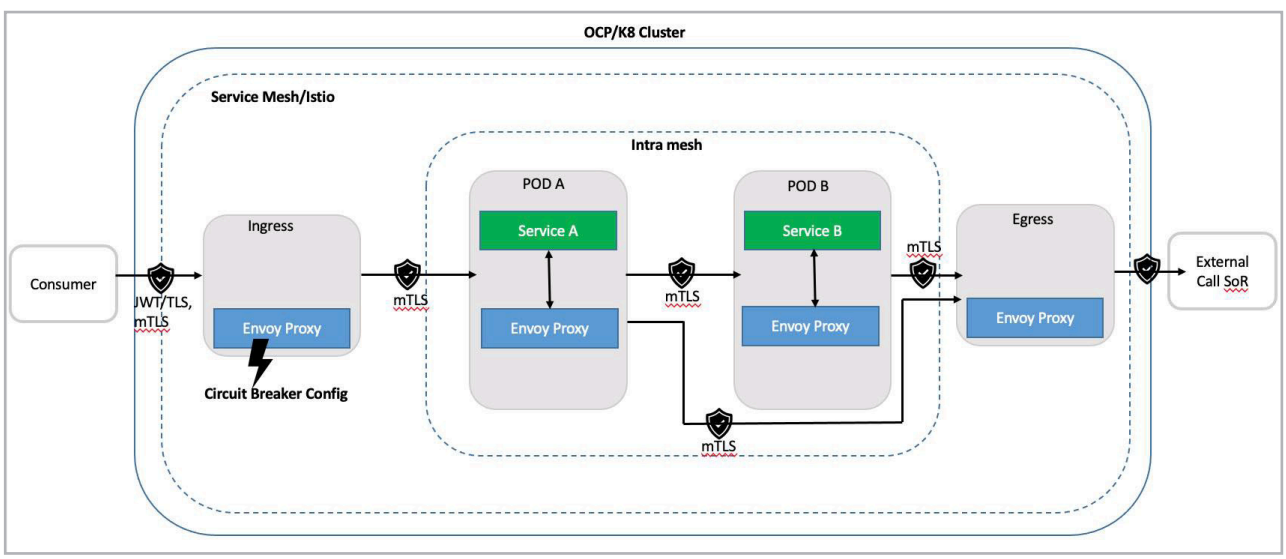

Figure 1: Circuit breaker at the point of ingress